



Overlay sandbox

A real browser host for exercising the chat overlay protocol.

Integration cases

Port 4300

Running chat required

`apps/chat-overlay-sandbox`

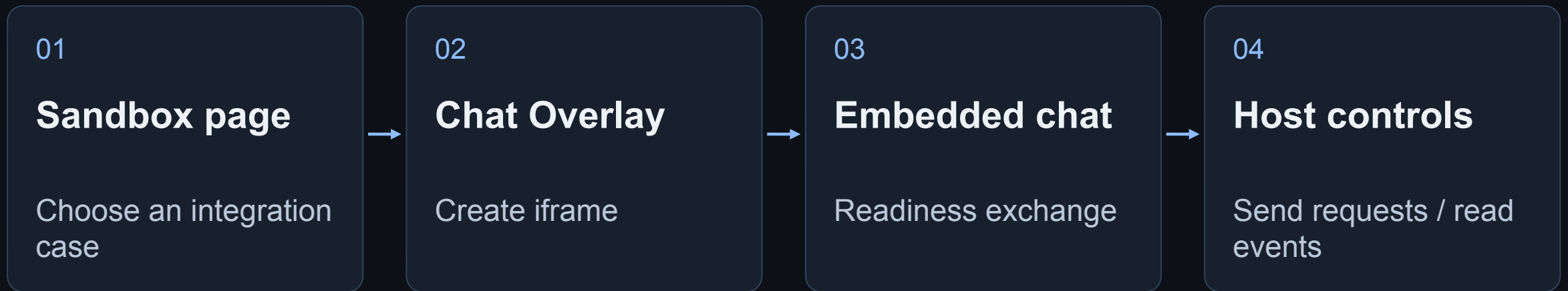


Each case demonstrates a distinct contract

Case	Demonstration
DirectOverlayCase	Mount into a host-owned region
ManagerOverlayCase	Floating widget controls
ConversationListCase	List, create, select, rename and delete
Features / auth cases	Live options and provider login UI modes



Trace the host-to-chat handshake



Host allowlisting and overlay mode are prerequisites.



Reproduce the local scenario

Configure the backend, then run both the chat and this sandbox.

```
OVERLAY_ENABLED=true
ALLOWED_IFRAME_ORIGINS=http://localhost:4300

# In the shell that starts the sandbox:
VITE_CHAT_OVERLAY_HOST=http://localhost:4207 \
  npm exec nx serve chat-overlay-sandbox
```



A direct overlay is a small integration

Checked pattern from [examples/chat-overlay.ts](#).

```
const overlay = new ChatOverlay('#demo-chat', {  
  domain: 'http://localhost:4207',  
  theme: 'dark',  
});  
await overlay.ready();  
await overlay.setInputContent('Explain the demo');  
// Later: overlay.destroy();
```



Deployment shares the main chat image

01

Published route

The backend can serve the sandbox at `/overlay-sandbox/`.

02

Enable it explicitly

The backend sandbox flag controls exposure of the route.

03

Separate HTML policy

The backend supplies the sandbox style nonce and response policy.



Verify cases through the current targets

Nx targets come from the resolved project graph.

```
npm exec nx test chat-overlay-sandbox  
npm exec nx build chat-overlay-sandbox  
npm exec nx run chat-overlay-sandbox:typecheck
```



Interpret integration failures at the right layer

01

Blank frame

Check chat CSP frame-ancestors and allowed host origin.

02

No ready event

Inspect protocol readiness after the iframe loads.

03

Login cannot complete

Verify the exact provider and configured UI mode.